

PIMES — Predictor de Interacciones de Medicamentos y Efectos Secundarios

Proyecto 1 de la asignatura *Inteligencia Artificial: Aprendizaje Automático*.

Este documento funciona como informe técnico del sistema **PIMES** (*Predictor de Interacciones de Medicamentos y Efectos Secundarios*). El objetivo del proyecto es estimar, para cada medicamento descrito por su vector binario de subestructuras químicas, la probabilidad de que produzca cada efecto secundario estudiado.

Información de los autores

- Autor 1: Jesús Gil — cédula 30175126
- Autor 2: Gabriel Castellano — cédula 28059781

Descripción del sistema

PIMES resuelve un problema de **clasificación multietiqueta**. Cada medicamento puede estar asociado con varios efectos secundarios al mismo tiempo, por lo que la salida no es una sola clase, sino un vector de probabilidades independiente por etiqueta.

Datos de entrada

El sistema trabaja con dos matrices tabulares:

- `medicamentos.txt`: matriz de 888 medicamentos representados por 881 subestructuras químicas binarias.
- `efectosSecundariosDeLosMedicamentos.txt`: matriz de 888 medicamentos por 1385 efectos secundarios binarios.

Además, en el modo de predicción se usa:

- `medicamentosSinEfectosSecun.txt`: medicamentos sin etiquetas conocidas.
- `listaDeEfectosSecun.txt`: lista de efectos secundarios a consultar, uno por línea.

Los archivos tabulares tienen un formato particular: la primera columna de cada fila contiene el nombre del medicamento, mientras que la primera fila contiene los nombres de las variables. El código usa `pandas` para interpretar ese desfase y reconstruir correctamente las matrices.

Arquitectura de la red neuronal

El núcleo del sistema es un `MLPClassifier` de `scikit-learn` con tres capas ocultas:

- Capa 1: 512 neuronas
- Capa 2: 256 neuronas
- Capa 3: 128 neuronas

La capa de entrada recibe 881 características binarias y la capa de salida produce 1385 probabilidades, una por efecto secundario. Se usa activación `ReLU` en las capas ocultas y un esquema sigmoide multietiqueta en la salida, lo que permite estimar la probabilidad de cada efecto de forma independiente.

Entrenamiento y validación

El sistema entrena y evalúa el modelo con **validación cruzada de 5 particiones**. En cada fold:

1. Se entrena una red neuronal sobre 4/5 de los datos.
2. Se predice sobre el fold de prueba.
3. Se calcula la curva ROC y el AUC micro-promedio.

Luego se concatenan todas las predicciones de validación para obtener una curva ROC global. Finalmente, el modelo se reentrena con todos los datos y se guarda en disco.

Parámetros utilizados

- Optimizador: `adam`
- Función de activación oculta: `relu`
- Tasa de aprendizaje inicial: `1e-3`
- Tamaño de lote: `32`
- Número máximo de iteraciones: `80`
- Semilla aleatoria: `42`
- Número de folds: `5`

Salidas del sistema

Al entrenar, el proyecto genera:

- `modelo_PIMES.joblib`: modelo final persistido junto con metadatos.
- `curva_ROC.png`: gráfica de la curva ROC micro-promediada.

En predicción, el sistema imprime para cada medicamento las probabilidades de los efectos secundarios solicitados, ordenadas de mayor a menor.

Factibilidad del proyecto

El proyecto es factible como sistema de predicción supervisada porque dispone de ejemplos etiquetados para entrenamiento y validación. Si además se cuentan con datos reales antiguos y sus efectos secundarios conocidos, esos datos pueden usarse como un conjunto de prueba externo para medir qué tan bien generaliza el modelo fuera del conjunto usado en el entrenamiento.

En un problema multietiqueta no basta con decir si "acertó" o no en forma global. Lo más útil es evaluar con varias métricas:

- `AUC micro-promedio`: resume el comportamiento general de todas las etiquetas.
- `precision@k`: verifica cuántos de los `k` efectos más probables realmente eran correctos.
- `recall@k`: mide cuántos efectos verdaderos aparecen entre los primeros `k` resultados.
- `F1 micro y macro`: permiten comparar precisión y cobertura en todas las etiquetas.
- `exact match`: sólo cuenta como acierto si coincide todo el vector de etiquetas, por lo que suele ser muy exigente.

Si los datos reales antiguos sólo tienen algunas etiquetas confirmadas, todavía se puede evaluar de forma parcial con `precision@k`, `recall@k` o revisión manual de los primeros resultados. Si tienen todas las etiquetas, entonces sí puede hacerse una evaluación cuantitativa completa y comparar el desempeño contra el entrenamiento original.

Requisitos

- Python 3.10 o superior
- Paquetes: `numpy`, `pandas`, `scikit-learn`, `matplotlib`, `joblib`

Instalación:

```
pip install numpy pandas scikit-learn matplotlib joblib
```

Uso

Entrenamiento

Desde la carpeta `Proy1_28059781_30175126`:

```
python PIMES.py  
-e
```

```
../datasets/datasets/medicamentos.txt
../datasets/datasets/efectosSecundariosDeLosMedicamentos.txt
```

Este modo realiza la validación cruzada, reporta el AUC y guarda el modelo entrenado.

Predicción

```
python PIMES.py
-p
../datasets/datasets/medicamentosSinEfectosSecun.txt
../datasets/datasets/listaDeEfectosSecun.txt
```

Este modo carga `modelo_PIMES.joblib` y genera las probabilidades de los efectos secundarios consultados.

Estructura del proyecto

```
Proy1_28059781_30175126/
|-- PIMES.py
|-- PIMES.ipynb
|-- README.md
|-- modelo_PIMES.joblib
+-- curva_ROC.png
```

Referencias bibliográficas

- Scikit-learn Developers. *MLPClassifier*. Documentación oficial de scikit-learn.
- Scikit-learn Developers. *Receiver Operating Characteristic (ROC) y Area Under the Curve (AUC)*. Documentación oficial de scikit-learn.
- Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825-2830.
- Chollet, F. (2017). *Deep Learning with Python*. Manning Publications.